FCEFyN
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Redes de Computadoras

Trabajo Práctico

N.º 5(Práctico)

Transmisión de Datos: de ICMP y ARP a Sockets TCP

Grupo: NetRunners
Docente: SANTIAGO MARTIN HENN

Integrantes

Bastida, Lucas Ramiro	DNI 39444511
Contessi, Ruben Andres	DNI 33315235
Garay, Ignacio Hernan	DNI 44191925
Giraudo, Juan Pablo	DNI 34970089
Peñaloza, Gonzalo Adrian	DNI 40441355
Quinteros del Castillo, Tomás Agustín	DNI 46169739
Vasconsellos Blason, Benjamin Alejandro	DNI 45642879

11 de octubre de 2026

1) ICMP y primer contacto con Wireshark

a) ICMP (Internet Control Message Protocol) es un protocolo auxiliar de la capa de red (Capa 3 del modelo OSI / Capa Internet de TCP/IP). Se utiliza para transmitir mensajes de control, diagnóstico, reporte de errores y pruebas de alcanzabilidad entre dispositivos de red (hosts y routers). ¿Transporta datos de aplicaciones? No. A diferencia de TCP o UDP, ICMP no está destinado al transporte de datos de usuario/aplicación final (como la navegación web, archivos o correo), sino al control y diagnóstico de la propia infraestructura de red.

b) El protocolo ICMP trabaja en la capa de red, pero viaja empaquetado/encapsulado dentro de IP. El receptor analiza el campo Protocol (Protocolo) de la cabecera IP. Cuando el valor de este campo es 1, la capa IP sabe que el payload (carga útil) contenido en el paquete IP es un mensaje ICMP.

c) ping es una herramienta de software que utiliza ICMP para verificar si un host destino es alcanzable a través de la red y medir el tiempo de ida y vuelta (Round Trip Time o RTT). Los campos que los distinguen son: Type (Tipo) y Code (Código) del encabezado ICMP.

d) Un mensaje ICMP tipo Echo contine:

- Type (1 byte): Identifica si es Request (8) o Reply (0).
- Code (1 byte): Subcódigo (generalmente 0 para Echo).
- Checksum (2 bytes): Para verificación de errores en la cabecera e información ICMP.
- Identifier (ID - 2 bytes): Para asociar solicitudes y respuestas en un proceso/proceso ping.
- Sequence Number (2 bytes): Número de secuencia para asociar cada Request específico con su Reply correspondiente.
- Data / Payload: Datos opcionales/de relleno (timestamps, secuencias de texto o números) enviados para calcular los tiempos y verificar que los datos lleguen intactos.

```
C:\Windows\system32>ping -4 192.168.100.1

Haciendo ping a 192.168.100.1 con 32 bytes de datos:
Respuesta desde 192.168.100.1: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.1: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.1: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.1: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 192.168.100.1:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 0ms, Máximo = 0ms, Media = 0ms

C:\Windows\system32>ping -4 8.8.8.8

Haciendo ping a 8.8.8.8 con 32 bytes de datos:
Respuesta desde 8.8.8.8: bytes=32 tiempo=15ms TTL=120
Respuesta desde 8.8.8.8: bytes=32 tiempo=15ms TTL=120
Respuesta desde 8.8.8.8: bytes=32 tiempo=16ms TTL=120
Respuesta desde 8.8.8.8: bytes=32 tiempo=15ms TTL=120

Estadísticas de ping para 8.8.8.8:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 15ms, Máximo = 16ms, Media = 15ms
```

Figure 1: Pings Gateway - 8.8.8.8

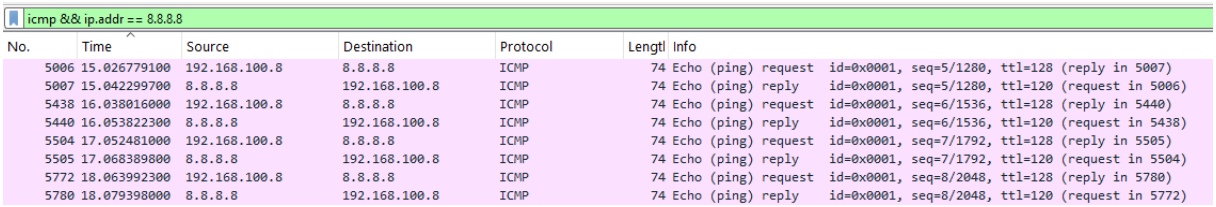


Figure 2: Captura WireShark con Filtro

e)

Capa (como la nombra Wireshark)	Origen	Destino	¿Qué campo indica qué protocolo viene "adentro"?
Ethernet II	fc:4d:a6:cd:4c:32	e0:d5:5e:2a:5e:ee	Type
Internet Protocol Version 4	192.168.100.8	8.8.8.8	Protocol
Internet Control Message Protocol	Identifier (BE) = 1 (0x0001)	Identifier (BE) = 1 (0x0001)	Type / Code
Datos / payload	N/A	N/A	N/A

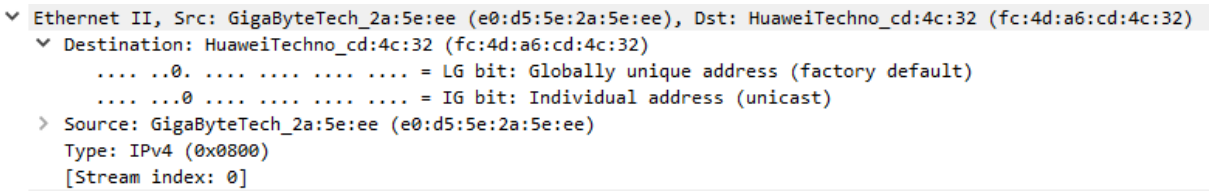


Figure 3: Echo Request 8.8.8.8

No, no es la MAC de 8.8.8.8. Es la dirección MAC de nuestro propio Router.

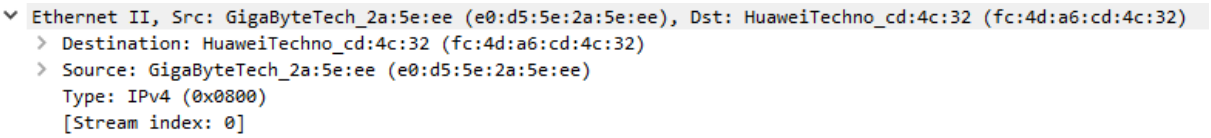


Figure 4: Echo Request al gateway

Como podemos observar en el echo Request al gateway, la dirección MAC de destino es la del router. De esto podemos concluir que el alcance de la direccion MAC es puramente local mientras que el alcance de la direccion IP es global por lo que para enviar un paquete a una dirección IP que no se encuentra en la misma red local, el paquete debe ser enviado al router, que se encargará de reenviarlo a su destino.

f) Los campos que cambian son:

- **Ethernet II:**
  - Source y Destination MAC Address: En este caso tiene sentido ya que uno es de request y el otro de reply, por lo que el sentido de flujo de datos se invierte.
- **IPv4:**
  - Identification: El identificador del paquete IP cambia en cada paquete enviado por la pila de red.
  - Source y Destination IP Address: Al igual que en el caso anterior, tiene sentido que se inviertan.
  - Time to Live (TTL).
  - Header Checksum: Cambia porque el TTL cambió, y el checksum se calcula sobre el header del paquete IP.

- **ICMP:**
  - Type: Cambia de 8 (echo request) a 0 (echo reply).
  - Checksum: Cambia porque el tipo cambió, y el checksum se calcula sobre todo el paquete ICMP.

Los campos que se mantienen igual son:

- **IPv4:** Version, Header Length, Protocol.
- **ICMP:** Identifier, Sequence Number.

Los campos identifier y sequence number se mantienen iguales porque le permiten a la herramienta ping asociar inequívocamente cada Echo Reply que llega con el Echo Request específico que envió previamente, calculando así el tiempo de ida y vuelta (RTT) exacto de cada paquete.

g) Se encuentra ubicado al final de la trama, como la carga útil empaquetada dentro del encabezado del protocolo ICMP. Tiene un tamaño de 32 bytes y contiene un patrón de bytes de relleno que consiste en una secuencia alfabética repetitiva de texto en formato ASCII y además es exactamente igual en el Reply.

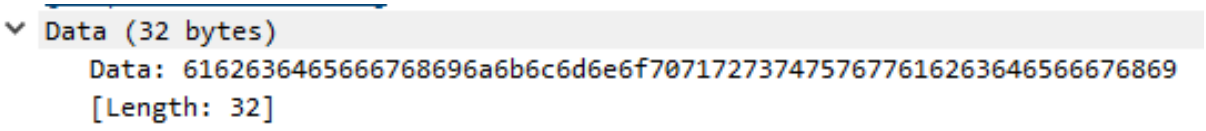


Figure 5: Payload del paquete ICMP.

Windows utiliza 32 bytes de letras ASCII mientras que Linux suele enviar 56 bytes incluyendo una marca de tiempo timestamp. Esto demuestra que la norma ICMP no exige un estándar fijo de contenido para el payload de prueba; cada sistema operativo implementa su propio generador de datos según el software ejecutable ping.

h) El echo request tiene un valor de Time to Live (TTL) de 128 y el del reply 120. Esto se debe a que cada router intermediario en Internet descuenta 1 unidad al campo TTL del paquete IP para evitar bucles de enrutamiento, dado que la respuesta enviada desde el servidor 8.8.8.8 se genera con un TTL = 128 el valor final recibido de 120 nos indica que la trama atravesó 8 saltos de routers intermedios hasta llegar de regreso a nuestro equipo.

i)

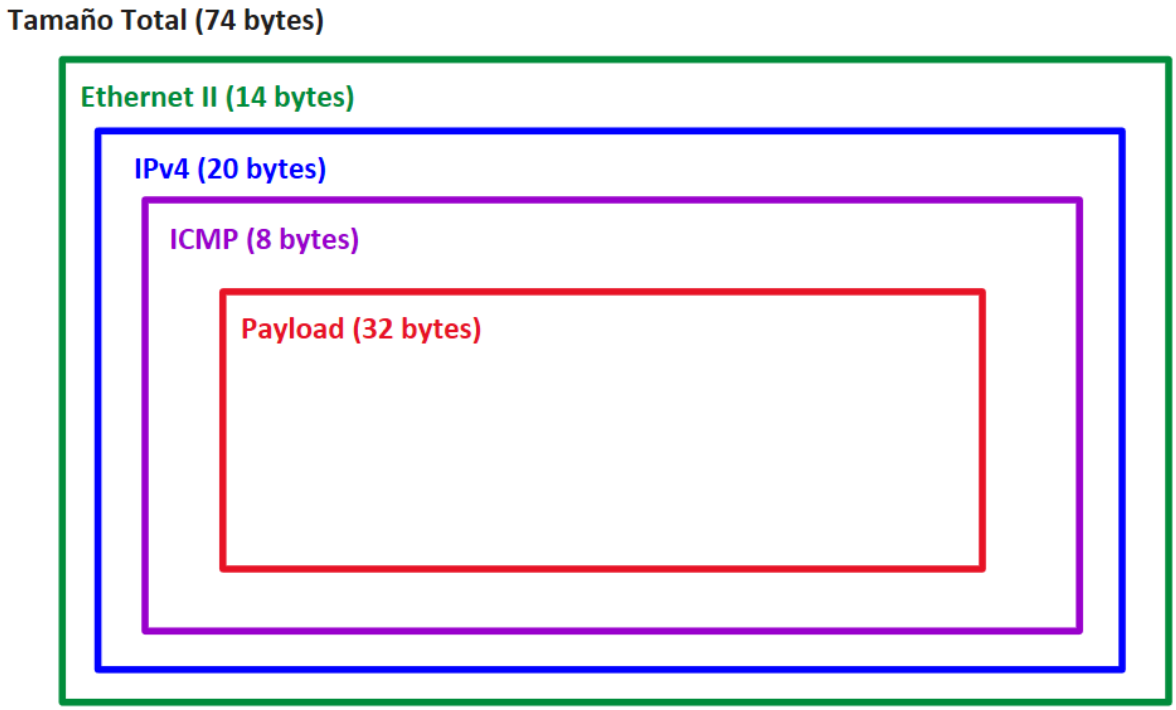


Figure 6: Encapsulado del paquete.

2) ARP: de una IP a una dirección MAC

a) ARP (Address Resolution Protocol) resuelve el problema de traducir una dirección IP (capa de red) a una dirección MAC (capa de enlace) dentro de una misma red local. Cuando una PC quiere mandarle un paquete a otra máquina de su subred, el sistema operativo sabe de antemano la IP destino, pero para meter esos datos en una trama Ethernet necesita la dirección MAC de destino; ARP es el mecanismo que se encarga de esto.

Ubicarlo en una capa es discutible porque no encaja bien en el modelo OSI de 7 capas ni en TCP/IP: "sirve" a la capa de red (resuelve direcciones IP), pero sus mensajes viajan directamente en tramas Ethernet, sin cabecera IP, es decir que opera al mismo nivel que la capa de enlace. El propio Stallings lo trata como un protocolo auxiliar en vez de como parte de una capa.

b) Un ARP Request es el mensaje de pregunta: "¿quién tiene la IP X? decime tu MAC". Se envía en broadcast Ethernet, porque el emisor todavía no sabe la MAC de nadie en particular. El ARP Reply es la respuesta: la manda únicamente la máquina que tiene esa IP, y se la envía en unicast directamente a la MAC del que preguntó (que ya venía incluida en el Request), porque sabe exactamente a quién contestarle.

c) La caché ARP es una tabla que cada sistema operativo mantiene en memoria con los pares IP-MAC que ya resolvió recientemente. Existe para evitar tener que hacer un Request/Reply por cada paquete que se manda: sin caché, cada trama hacia un mismo destino implicaría un broadcast ARP previo, generando tráfico innecesario y demoras. Con la caché, una vez resuelta una IP, las tramas siguientes hacia esa misma MAC se arman directamente sin volver a preguntar, hasta que la entrada expira.

d) Primero se consulta la caché ARP local: si ya hay una entrada para esa IP, se usa esa MAC directamente. Si no está, el sistema operativo dispara un ARP Request preguntando por esa IP; todas las máquinas del segmento lo reciben, pero solo la dueña responde con un ARP Reply indicando su MAC. Con esa respuesta se arma la trama Ethernet con la MAC destino correspondiente, y se guarda en la caché para no tener que repetir el proceso la próxima vez.

e) Sí, la dirección MAC coincide con la que vimos anteriormente en el punto 1.

f) Luego de hacer la captura, habiendo elegido la opción A, haciendo ping a 192.168.1.253 (usamos esta IP ya que nuestra subred es 192.168.1.0/24 con gateway 192.168.1.1, entonces usamos una IP que caiga en el rango; aclaramos que realizamos este punto en otra PC distinta a la del punto 1), podemos observar:

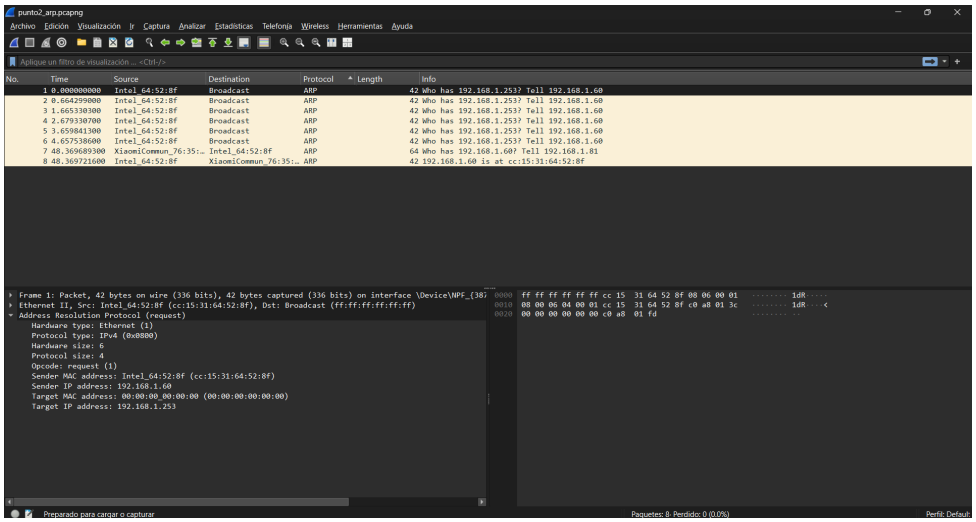


Figure 7: Captura de Wireshark: ARP Requests a 192.168.1.253 sin Reply

La tabla del Request quedaría:

- MAC destino (Ethernet): ff:ff:ff:ff:ff:ff (Broadcast)
- MAC origen (Ethernet): cc:15:31:64:52:8f (Intel_64:52:8f, nuestra placa)
- Opcode: 1 (request)
- Sender MAC address: cc:15:31:64:52:8f
- Sender IP address: 192.168.1.60
- Target MAC address: 00:00:00:00:00:00
- Target IP address: 192.168.1.253

Mientras que la tabla del Reply queda vacía, ¿por qué ocurre esto? Son todos ARP Request idénticos, de la PC hacia Broadcast preguntando por 192.168.1.253, y ninguno tiene su correspondiente Reply. Eso nos dice que como esa IP no existe, nadie contesta. Respondiendo a las preguntas derivadas de esto:

a) El Request va a broadcast porque nadie sabe todavía quién tiene esa IP, y el Target MAC address aparece en 00:00:00:00:00:00 porque es justamente el dato que se busca.

b) Después de las dos direcciones MAC, aparece 08 06; ese es el campo Type de Ethernet, y vale 0x0806, el EtherType reservado para ARP. No hay encabezado IP en el medio; esto nos dice que ARP viaja directo sobre Ethernet, no dentro de IP.

c) Aparecieron 6 ARP Request, ningún Reply y ningún ICMP Echo Request en la captura, y esto es porque sin la MAC resuelta, el sistema operativo nunca llega a armar la trama del ping.

d) No hizo falta esperar el minuto: como 192.168.1.253 nunca nos contestó, nunca se guardó nada en la caché para esa IP (eso solo pasa cuando hay Reply), así que daba lo mismo pingear de nuevo al instante o esperar. Repetimos el ping y, tal cual esperábamos, salió otro ARP Request en broadcast, sin respuesta; lo confirmamos también con `arp -a`.

La ventaja de la caché es justamente esa: evita mandar un ARP por cada paquete que se manda a un destino ya conocido. El problema es que si una entrada queda vieja y la MAC real cambió, se van a seguir mandando tramas a una MAC que ya no corresponde, hasta que esa entrada expire sola.

3) TCP y UDP a mano con ncat

Conceptos previos

a) **¿Qué significa establecer una conexión? ¿Dónde existe una conexión TCP?**

Establecer una conexión TCP significa realizar un intercambio inicial de segmentos entre los extremos para iniciar la comunicación. Este establecimiento se realiza mediante tres pasos: SYN, SYN-ACK y ACK. La conexión es una asociación lógica entre los extremos, no algo que exista en los cables o routers.

b) **¿Qué es un puerto? ¿Qué identifica el par (IP, puerto)?**

Un puerto es un punto de acceso que permite identificar un proceso o servicio dentro de un equipo. El par (IP, puerto) permite identificar un extremo de comunicación, indicando el equipo y el proceso o servicio al que se dirige el tráfico.

c) **¿Qué significa que un proceso esté escuchando en un puerto?**

Significa que el proceso está preparado para recibir comunicaciones dirigidas a ese puerto. En TCP, el servidor espera solicitudes de conexión de los clientes para poder establecer una conexión y comenzar el intercambio de datos.

Experimento TCP

Se ejecutó un servidor con `ncat` escuchando en el puerto TCP 12000 y un cliente que se conectó a la dirección `127.0.0.1`. La captura se realizó con Wireshark utilizando el filtro `tcp.port == 12000`.

En la captura se identificaron las siguientes etapas:

- **Establecimiento de la conexión:** se observan los segmentos SYN, SYN-ACK y ACK, que conforman el establecimiento de la conexión TCP.
- **Intercambio de datos:** aparecen segmentos con las banderas PSH y ACK que transportan los mensajes enviados entre el cliente y el servidor. También se observan segmentos ACK sin datos de aplicación.
- **Cierre de la conexión:** se identifican segmentos FIN-ACK y ACK que intervienen en el cierre de la conexión.

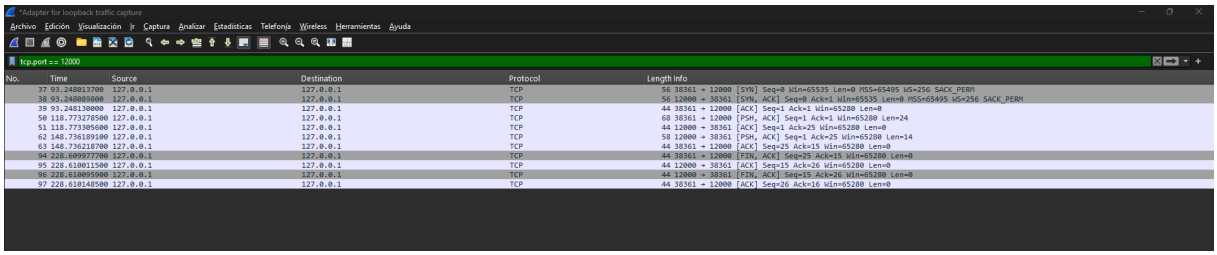


Figure 8: Captura del intercambio TCP con Wireshark.

Experimento UDP

Se realizó una comunicación UDP utilizando el puerto 12001. El servidor quedó a la espera de datagramas y el cliente envió el mensaje "Hola servidor". Luego, el servidor respondió con el mensaje "Hola cliente". El tráfico se capturó con Wireshark.

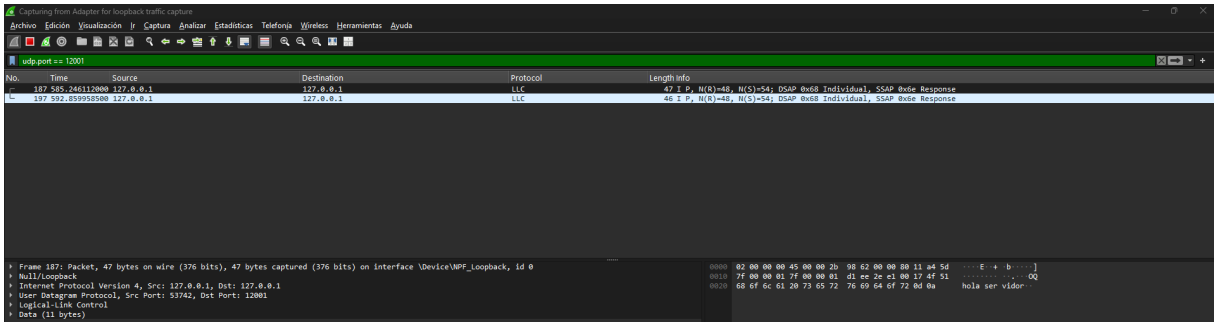


Figure 9: Datagrama UDP enviado desde el cliente al servidor.

En esta captura se observa un datagrama con puerto de origen 53742 y puerto de destino 12001. Los datos transmitidos ocupan 11 bytes y corresponden al mensaje "Hola servidor".

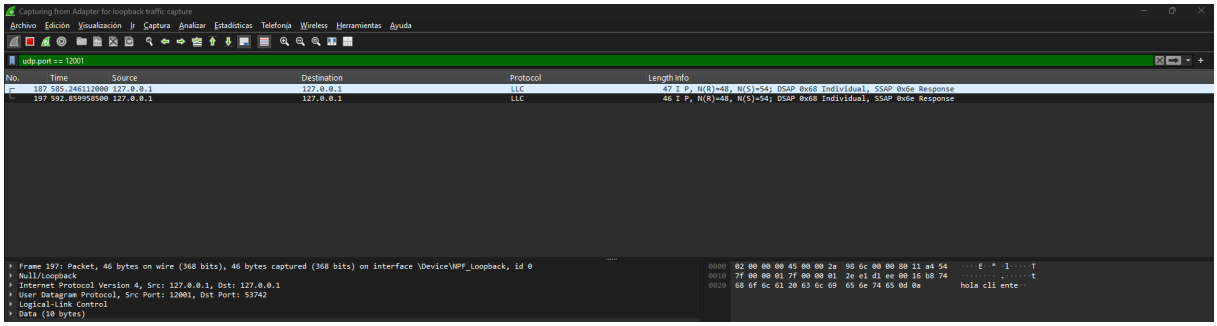


Figure 10: Datagrama UDP enviado desde el servidor al cliente.

En esta captura se observa el sentido inverso: el puerto de origen es 12001 y el puerto de destino es 53742. Los datos transmitidos ocupan 10 bytes y corresponden al mensaje "Hola cliente".

En ambas capturas, aunque la columna *Protocol* de Wireshark muestra LLC, el panel de detalles permite identificar los encabezados IPv4 y UDP, junto con los puertos de origen y destino.

Comparación entre TCP y UDP

a) Antes de enviar el primer mensaje, UDP no necesita establecer una conexión. En cambio, TCP realiza un intercambio inicial de tres segmentos (SYN, SYN-ACK y ACK) antes de

transmitir los datos.

b) En la captura UDP, cada uno de los mensajes observados se transmite en un datagrama independiente. No se observan confirmaciones ACK propias de UDP. En TCP, en cambio, se observan segmentos de confirmación durante el intercambio de datos.

c) Comparación de las cabeceras TCP y UDP.

La cabecera TCP observada en Wireshark tiene un tamaño de 20 bytes (160 bits), mientras que la cabecera UDP tiene un tamaño fijo de 8 bytes (64 bits). Ambos protocolos incluyen los puertos de origen y destino. TCP incorpora además campos como números de secuencia, confirmación y banderas de control, mientras que UDP posee una cabecera más simple, con campos de longitud y suma de comprobación.

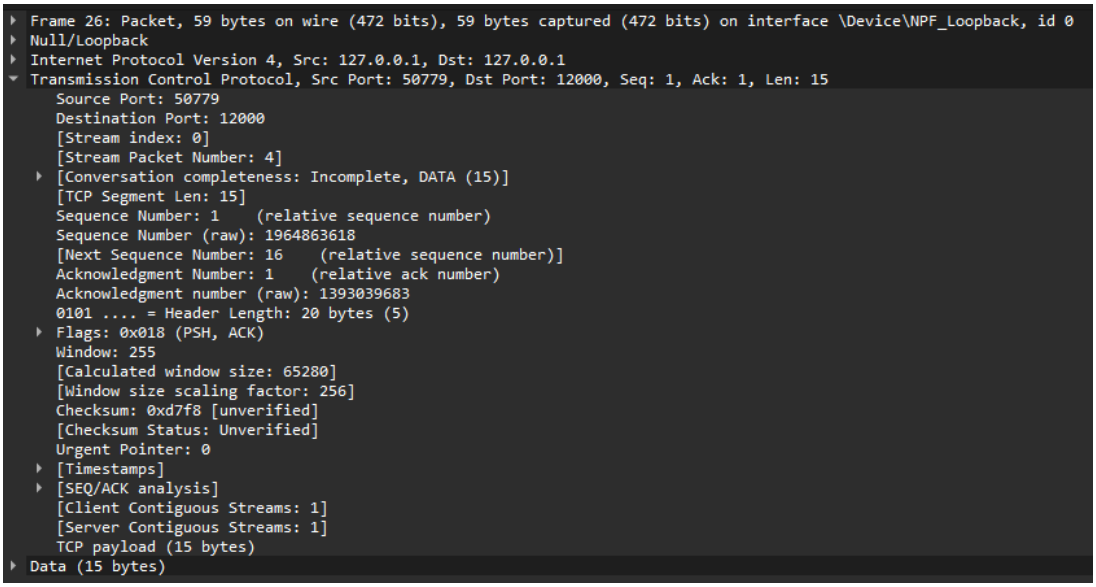


Figure 11: Detalle de la cabecera TCP en Wireshark.

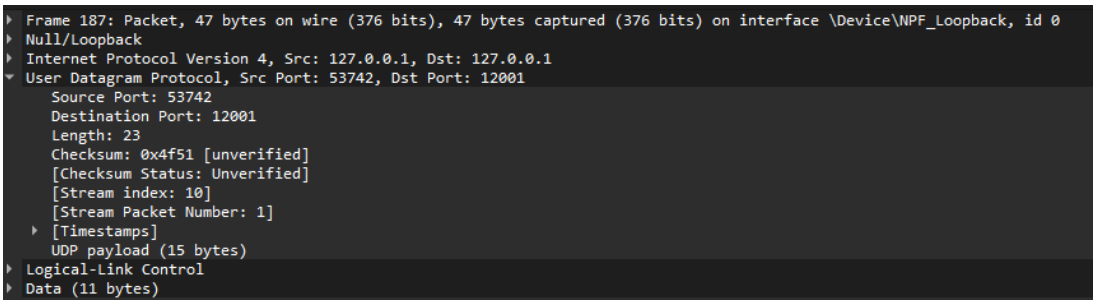


Figure 12: Detalle de la cabecera UDP en Wireshark.

d) Al cerrar el cliente UDP con Ctrl+C, no se realiza un intercambio de mensajes de cierre, ya que UDP no establece una conexión. En cambio, al cerrar la conexión TCP se observa un intercambio de segmentos FIN y ACK para finalizar la comunicación de forma ordenada.

e) En ambas pruebas se envió el mensaje “hola” desde el cliente hacia el servidor.

En la captura TCP se observan 9 paquetes en total: 3 para establecer la conexión, 2 correspondientes al envío del mensaje y su confirmación, y 4 para cerrar la conexión. En la captura UDP se observa un datagrama para enviar el mensaje.

Los paquetes adicionales de TCP permiten establecer y cerrar la conexión, además de confirmar la recepción de los datos. Estos mecanismos contribuyen a ofrecer una comunicación confiable y ordenada, mientras que UDP no requiere establecer una conexión ni utiliza confirmaciones de recepción propias del protocolo.

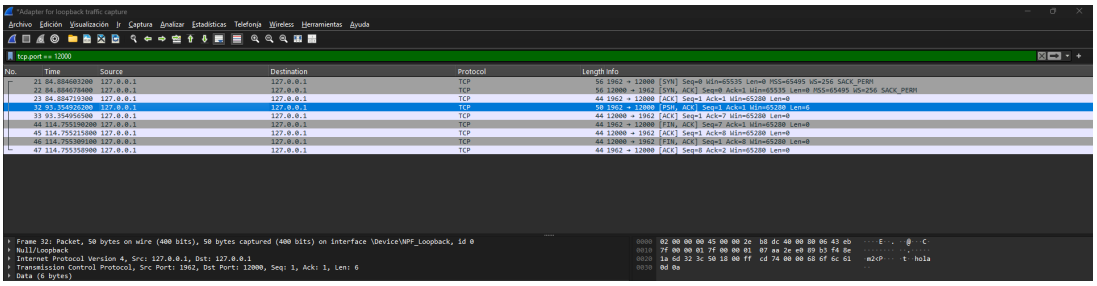


Figure 13: Captura TCP del envío del mensaje "hola" desde el cliente al servidor, incluyendo el establecimiento y el cierre de la conexión.

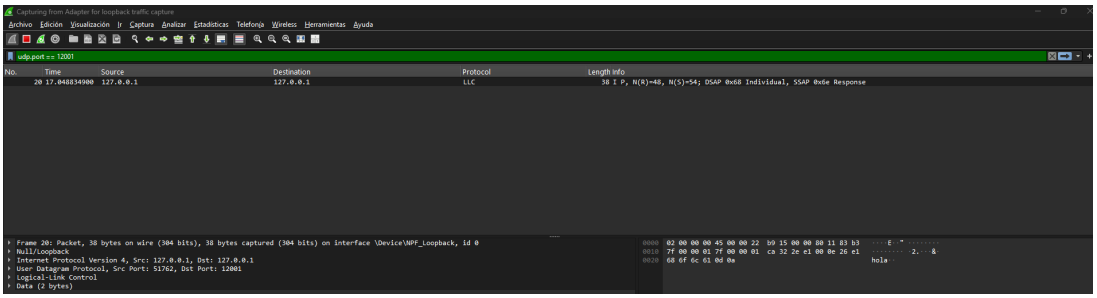


Figure 14: Captura UDP del envío del mensaje "hola" desde el cliente al servidor.

f) Al intentar establecer una conexión TCP con el puerto 12000, el cliente envía segmentos SYN y recibe respuestas RST, ACK, indicando que la conexión es rechazada porque no hay ningún proceso escuchando en ese puerto.

En el caso de UDP, el cliente puede enviar un datagrama al puerto 12001 sin establecer previamente una conexión. Como no hay ningún proceso escuchando en ese puerto, se recibe un mensaje ICMP *Destination unreachable (Port unreachable)*, que indica que el puerto de destino no está disponible.

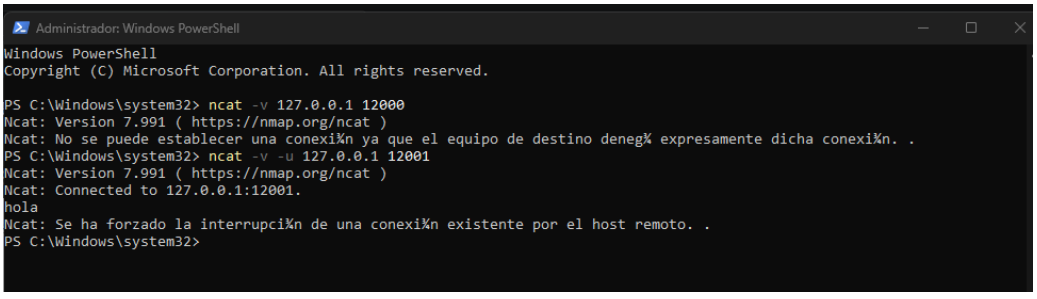


Figure 15: Salida de PowerShell al intentar comunicarse con los puertos TCP y UDP sin un servidor escuchando.

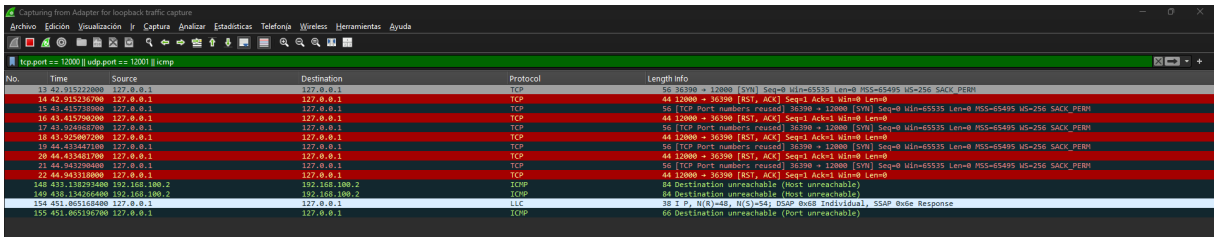


Figure 16: Captura de Wireshark con los intentos de conexión TCP y el mensaje ICMP de puerto UDP inaccesible.

4) Servidor TCP minimo

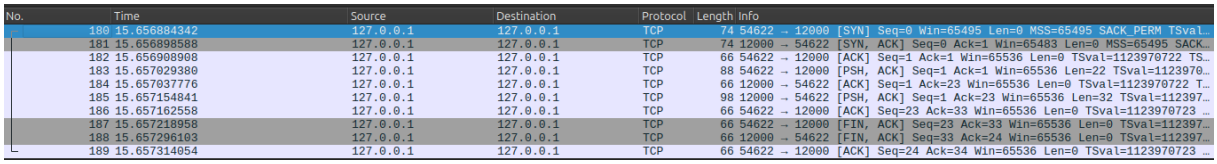


Figure 17: Captura de la ejecución del servidor y cliente TCP.

a) **Salida de consola del servidor:**

```
python3 tcp_server.py
[13:07:33.301010] [servidor TCP] escuchando en 127.0.0.1:12000 ...
[13:07:41.866240] [servidor TCP] conexión aceptada desde 127.0.0.1:54622
[13:07:41.866317] [servidor TCP] recibido (22 bytes): 'Hola desde mi programa'
[13:07:41.866433] [servidor TCP] respuesta enviada: 'Recibido: Hola desde mi
↪  programa'
[13:07:41.866505] [servidor TCP] el cliente cerró la conexión
```

Salida de consola del cliente:

```
python3 tcp_client.py 127.0.0.1 "Hola desde mi programa"
[13:07:41.866208] [cliente TCP] conectado a 127.0.0.1:12000 desde
↪  127.0.0.1:54622
[13:07:41.866309] [cliente TCP] enviado: 'Hola desde mi programa'
[13:07:41.866438] [cliente TCP] respuesta (32 bytes): 'Recibido: Hola desde mi
↪  programa'
```

d) Para cada llamada, se indica si genera tráfico y, en caso afirmativo, los segmentos observados en la captura.

Llamada	¿Dónde se ejecuta?	¿Genera tráfico?	Segmentos que se observan
`socket()`	Servidor y cliente	No	–
`bind()`	Servidor	No	–
`listen()`	Servidor	No	–
`connect()`	Cliente	Sí	N.º 180: SYN (cliente a servidor), N.º 181: SYN, ACK (servidor a cliente) y N.º 182: ACK (cliente a servidor)
`accept()`	Servidor	No	–
`sendall()`	Servidor	Sí	N.º 185: PSH, ACK con la respuesta; N.º 186: ACK del cliente
`recv()`	Servidor	No	–
`close()`	Ambos	Sí	N.º 187: FIN, ACK del cliente; N.º 188: FIN, ACK del servidor; N.º 189: ACK final del cliente

Table 1: Relación entre las llamadas de socket y el tráfico TCP observado.